

RESUMEN TEMA 1 AL 4

Apuntes del Tema 1: Administración de Bases de Datos

1. Conceptos Básicos y Arquitectura

- **Arquitectura Cliente/Servidor:** Las aplicaciones de bases de datos operan bajo esta arquitectura, que puede ser centralizada o distribuida.
- **Base de Datos vs. Instancia:** Es fundamental comprender esta distinción en Oracle:
 - **Base de datos:** Es la estructura física; es decir, el conjunto de ficheros de datos y de configuración almacenados en discos,.
 - **Instancia:** Es la estructura lógica que se encuentra en la memoria; consiste en un conjunto de procesos y áreas de memoria compartida que gestionan la base de datos,.
 - Normalmente hay una correspondencia de uno a uno (una instancia para una base de datos), aunque pueden existir configuraciones de muchas instancias para una sola base de datos, como en *Real Applications Cluster* (RAC).

2. Estructura Física: La Base de Datos La base de datos está compuesta por diferentes tipos de archivos para su funcionamiento y gestión:

- **Ficheros de software:** Se localizan en el directorio ORACLE_HOME e incluyen el software necesario tanto en el cliente como en el servidor.
- **Ficheros de configuración:** Destacan el archivo de inicialización (pfile en texto o spfile en binario, comúnmente llamado init.ora) utilizado en el arranque, el listener.ora que gestiona las conexiones, y el tnsnames.ora que contiene la información de localización,.
- **Ficheros de datos (Datafiles):** Almacenan la información real. Se organizan en agrupaciones lógicas llamadas **tablespaces** (como SYSTEM, SYSAUX, USERS, TEMP, UNDO), y la información dentro de ellos se divide en bloques o páginas de tamaños fijos (frecuentemente de 8 Kbytes),.
- **Redo log files:** Registran las transacciones (información original o *undo*, e información nueva o *redo*) para permitir deshacer o rehacer operaciones y recuperar el sistema ante fallos. Cuando se llenan, el sistema realiza copias denominadas **Archived Redo log files**.

- **Otros ficheros:** Ficheros temporales para resultados intermedios, fichero de control para el arranque del sistema, *Event logs* para registrar eventos importantes y archivos de *Backups* o copias de seguridad,.

3. Estructura Lógica: La Instancia La instancia se compone de memoria compartida y procesos clave:

- **SGA (System Global Area):** Es un gran espacio en la memoria principal compartido por los procesos del servidor. Puede configurarse mediante el parámetro MEMORY_TARGET. Sus componentes principales son:
 - **Buffer cache:** Almacena bloques de datos traídos del disco para mejorar el rendimiento; utiliza un algoritmo de reemplazo para los bloques menos recientemente usados (LRU).
 - **Shared Pool:** Guarda los planes de ejecución de consultas (shared SQL area) para reutilizarlos, así como la caché del diccionario de datos.
 - **Log Buffer:** Mantiene una cola de las entradas de transacciones (rehacer/deshacer) que luego serán pasadas a disco.
- **Procesos de primer plano (Foreground):** Se inician cuando un usuario se conecta a la base de datos. Validan los permisos del usuario, generan los planes de ejecución de sus consultas y manipulan los bloques de datos.
- **Procesos de segundo plano (Background):** Procesos que operan constantemente para el funcionamiento interno. Los más importantes son:
 - **DBWR (Database writer):** Escribe los bloques de datos modificados desde la caché a los discos físicos.
 - **LGWR (Log writer):** Transfiere la información del *Log Buffer* a los archivos físicos de *Redo log*.
 - **ARCH (Archiver):** Genera copias de los ficheros *Redo log* llenos.
 - **PMON (Process monitor):** Supervisa el estado de las conexiones y limpia aquellas que se han caído.
 - **SMON (System monitor):** Realiza limpieza y recuperación tras paradas inesperadas.
 - **CKPT (Checkpoint):** Obliga periódicamente a transferir los bloques modificados a disco.

4. Administración de Usuarios y Esquemas Las tareas principales del Administrador de la Base de Datos (DBA) implican controlar los accesos y la estructura:

- **Creación y control de Usuarios:** Se pueden crear cuentas con `CREATE USER <usuario> IDENTIFIED BY <contraseña>`. Un usuario puede ser bloqueado o desbloqueado posteriormente con `ALTER USER <usuario> ACCOUNT LOCK/UNLOCK`, y su contraseña puede actualizarse.
- **Privilegios:** Un usuario recién creado requiere permisos explícitos otorgados mediante `GRANT` (por ejemplo, `GRANT CONNECT` para acceder al sistema y `GRANT RESOURCE` para crear tablas).
- **Esquemas:** Un esquema está directamente relacionado con un usuario y actúa como el contenedor lógico de sus objetos (tablas, vistas, etc.). Se puede consultar qué tablas hay en un esquema, permitir a otros usuarios leer esos datos mediante `GRANT SELECT ON <esquema.tabla> TO <usuario>`, o eliminar completamente un esquema junto con sus objetos usando `DROP USER <esquema> CASCADE`.

5. Monitoreo y Mantenimiento El control de la base de datos se realiza consultando tablas dinámicas (diccionario de datos):

- **Sesiones activas:** Mediante `SELECT username, machine FROM v$session`; se puede identificar qué usuarios están conectados.
- **Consumo de memoria:** A través de `SELECT * FROM v$sga`; se revisa el tamaño y utilización del área compartida SGA.
- **Errores del sistema:** Para verificar alertas e incidentes, el DBA debe revisar el log de eventos usando `SELECT * FROM v$alert_log ORDER BY timestamp DESC`;
- **Rendimiento:** Parte de las labores incluyen optimizar consultas mediante la creación de índices con instrucciones como `CREATE INDEX ....`

Apuntes del Tema 2: Bases de Datos de Objetos (BDO) y Objeto-Relacionales (BDOR)

1. Bases de Datos de Objetos (BDO) Las BDO están diseñadas para almacenar objetos de forma nativa, manteniendo su estructura y su comportamiento, lo que

facilita enormemente la integración con lenguajes de programación orientados a objetos.

- **Conceptos fundamentales:**

- **Objeto:** Formado por un estado (valores o atributos) y un comportamiento (operaciones o métodos).
- **Identidad de objeto (OID):** Cada objeto persistente recibe un identificador único asignado por la base de datos, lo que permite diferenciarlos incluso si sus valores son idénticos.
- **Encapsulamiento y persistencia:** Los objetos pueden ocultar detalles internos exponiendo solo una interfaz. La persistencia permite que los objetos se almacenen de forma permanente más allá de la ejecución de una aplicación.
- **Jerarquías y herencia:** Permite definir nuevos tipos basados en otros predefinidos, heredando sus atributos y métodos. Se permite la herencia múltiple (un subtipo hereda de dos o más supertipos), aunque puede generar problemas de ambigüedad que deben resolverse.
- **Polimorfismo (Sobrecarga de operadores):** Un mismo nombre de operación puede tener múltiples implementaciones dependiendo del tipo de objeto sobre el que se aplique.

2. Bases de Datos Objeto-Relacionales (BDOR) Combinan la robustez de las bases de datos relacionales tradicionales con las capacidades del modelo orientado a objetos. Se basan en extensiones del estándar SQL (como SQL:99 o SQL/Object) para incorporar tipos definidos por el usuario, tipos colección y referencias.

3. Implementación y Definición de Tipos en Oracle En Oracle, se separa la definición de los tipos de la creación física de las tablas:

- **Creación de Tipos:** Se utiliza la instrucción `CREATE TYPE <nombre> AS OBJECT (...)` para definir la estructura de un objeto (clase).
- **Creación de Tablas:** Se pueden crear tablas compuestas enteramente por objetos utilizando la sintaxis `CREATE TABLE <nombre_tabla> OF <Tipo_Objeto>`. Alternativamente, un tipo de objeto puede usarse como una simple columna dentro de una tabla relacional convencional.
- **Consultas:** Al consultar atributos de un objeto almacenado en una tabla relacional usando la notación por puntos (ej. `tabla.atributo`), es obligatorio utilizar un alias de tabla (ej. `SELECT c.nombre FROM Clientes c;`).

4. Herencia en Oracle

- Para que un tipo pueda tener subtipos, el supertipo debe definirse añadiendo la cláusula **NOT FINAL**.
- Los subtipos se crean con la palabra clave **UNDER**, heredando automáticamente del tipo padre.
- Una tabla creada a partir de un supertipo puede almacenar tanto objetos del supertipo como de cualquiera de sus subtipos.
- Para consultar o acceder a los atributos específicos de un subtipo en una tabla genérica, se utiliza la función **TREAT** (ej. `SELECT TREAT(VALUE(c) AS ClienteVIP).descuento FROM Clientes c;`).

5. Relaciones y Colecciones

- **Referencias (REF):** Permiten relacionar objetos sin usar claves foráneas tradicionales. Un atributo se define de tipo REF hacia otro tipo de objeto.
 - Se utiliza **SCOPE IS** en la definición de la tabla para restringir que la referencia apunte obligatoriamente a los objetos de una tabla concreta.
 - Para acceder al objeto o a los atributos referenciados se utiliza la función **DEREF**.
- **Colecciones:**
 - **VARRAY:** Vectores de tamaño fijo que almacenan múltiples elementos en un solo atributo.
 - **Tablas Nidadas (Nested Tables):** Colecciones similares pero sin límite predefinido de tamaño y sin un orden estricto de elementos.

6. Métodos y Constructores Los tipos definidos por el usuario pueden encapsular operaciones:

- En la definición del objeto (`CREATE TYPE`) se declaran las funciones o procedimientos con **MEMBER FUNCTION** o **MEMBER PROCEDURE**.
- La lógica y el código en PL/SQL se implementan utilizando **CREATE TYPE BODY**. Dentro del código del método, se puede usar **SELF** para hacer referencia al objeto actual que invoca el método.
- Aunque existe un constructor por defecto, se pueden definir constructores personalizados mediante **CONSTRUCTOR FUNCTION**, los cuales deben retornar **SELF AS RESULT**.

Apuntes del Tema 3: PL/SQL (Procedural Language/SQL)

1. Introducción y Tipos de Bloques PL/SQL es un lenguaje de programación estructurado y procedural que extiende las capacidades de SQL en las bases de datos Oracle. Su gran ventaja es que permite ejecutar múltiples instrucciones SQL en un solo bloque y utilizar lógica de programación como condicionales y bucles. El código se organiza en distintos tipos de bloques:

- **Bloques anónimos:** Se ejecutan de forma independiente y no se almacenan de forma permanente en la base de datos.
- **Procedimientos almacenados:** Se guardan en la base de datos para ser llamados repetidamente. No devuelven un valor directo (con un RETURN), sino que se utilizan para ejecutar acciones (ej. actualizaciones) y se comunican mediante parámetros de entrada (IN), salida (OUT) o ambos (IN OUT).
- **Funciones almacenadas:** Son similares a los procedimientos, pero obligatoriamente retornan un valor específico. Tienen la ventaja de que pueden ser utilizadas directamente dentro de sentencias SQL.

2. Estructura Básica y Variables La estructura general de un bloque PL/SQL se compone de las secciones DECLARE (opcional, para variables), BEGIN (instrucciones), EXCEPTION (opcional, para errores) y END;.

- **Asignación de variables:** Se utiliza el operador := (ej. v_bono := 1500;).
- **Consultas a variables:** Mediante la cláusula INTO, se puede asignar el resultado de una consulta SQL directamente a una variable (ej. SELECT salario INTO v_salario FROM...).
- **Operadores:** Se soportan operadores aritméticos (+, -, *, /, **), relacionales (=, <, >, <=, >=, <>), lógicos (AND, OR, NOT) y de concatenación de cadenas (||).
- **Salida por pantalla:** Para mostrar mensajes se utiliza DBMS_OUTPUT.PUT_LINE(...) (requiere tener activo el parámetro SET SERVEROUTPUT ON).

3. Estructuras de Control Condicionales PL/SQL permite modificar el flujo de ejecución mediante condiciones:

- **IF-THEN / IF-THEN-ELSE:** Ejecuta un bloque de código si la condición es verdadera, y opcionalmente otro si es falsa.
- **IF-THEN-ELSIF:** Permite evaluar múltiples condiciones de forma encadenada.

- **CASE-WHEN:** Es una estructura más limpia y legible cuando se deben evaluar múltiples valores discretos de una misma variable.

4. Estructuras Iterativas (Bucles) Sirven para repetir un conjunto de instrucciones de manera controlada:

- **LOOP:** Es el bucle básico. Se ejecuta de forma indefinida hasta que encuentra una instrucción de salida manual EXIT o condicional EXIT WHEN.
- **WHILE LOOP:** Repite las instrucciones siempre y cuando se cumpla una condición lógica definida al inicio.
- **FOR LOOP:** Incorpora un contador automático que itera sobre un rango de valores fijo (ej. FOR i IN 1..10 LOOP).

5. Cursores Los cursores son esenciales para recorrer y trabajar con consultas que devuelven múltiples filas, procesándolas registro por registro.

- **Cursores explícitos:** Se deben controlar manualmente. Requieren ser declarados (DECLARE CURSOR c IS ...), abiertos (OPEN c), recorridos fila a fila extrayendo los datos a variables (FETCH c INTO ...), verificar si ya no hay datos (EXIT WHEN c%NOTFOUND) y finalmente cerrarlos (CLOSE c).
- **Cursores implícitos (FOR):** Simplifican el código, ya que permiten usar un bucle FOR directamente sobre una consulta SELECT, abriendo, recorriendo y cerrando el cursor automáticamente.

6. Manejo de Excepciones El bloque EXCEPTION sirve para capturar y gestionar los errores en tiempo de ejecución, evitando que el programa se caiga de forma abrupta. Las excepciones más comunes son:

- **NO_DATA_FOUND:** Ocurre cuando una consulta SELECT ... INTO no devuelve ninguna fila.
- **TOO_MANY_ROWS:** Se dispara cuando una consulta con INTO devuelve más de un registro, lo cual no está permitido en variables simples.
- **ZERO_DIVIDE:** Ocurre al intentar dividir un número entre cero.

7. Gestión de Subprogramas y SQL Dinámico

- **DDL para Subprogramas:** Los procedimientos y funciones se crean con CREATE, se modifican con REPLACE (usualmente se combinan como CREATE OR REPLACE) y se eliminan de la base de datos usando DROP.
- **SQL Dinámico:** Si se necesita ejecutar código construido en tiempo de ejecución (por ejemplo, con nombres de tablas o columnas pasados por parámetro), se utiliza la instrucción **EXECUTE IMMEDIATE**. Esto permite

construir la consulta como un texto (VARCHAR) y ejecutarla dinámicamente pasándole valores con la cláusula USING.

Apuntes del Tema 4: Continguts Multimèdia (Contenidos Multimedia)

1. Características y Requisitos de los Datos Multimèdia Los datos multimedia engloban **imágenes, texto con formato, vídeo y audio**. Manejar estos datos en una base de datos presenta retos específicos:

- **Mida elevada:** Requieren mucho espacio de almacenamiento (desde Megabytes hasta Gigabytes) y tienen un coste elevado tanto de procesamiento como de transmisión.
- **Información temporal:** Suelen tener un orden secuencial que debe respetarse de forma estricta (por ejemplo, mostrar los *frames* de un vídeo en el orden correcto).
- **Semántica compleja:** A diferencia de los datos tradicionales, es mucho más difícil describir qué características definen, por ejemplo, una imagen.

2. Metadatos Para solucionar la dificultad de consultar directamente los datos multimedia, se utilizan **metadatos**, que son datos adicionales necesarios para interpretar el significado del contenido principal.

- **Ejemplos de metadatos:** La textura o el histograma de una imagen, las frecuencias o la identificación de quién habla en un fragmento de audio, la dirección de movimiento en un vídeo, o el tipo de fuente de un texto.
- El proceso de generar metadatos es costoso, por lo que es ideal que sea automatizado. Existen tres formas de generarlos:
 - **Manualmente:** Definir la estructura de un documento, características de imágenes o marcar los segmentos de un vídeo/audio.
 - **Semi-automáticamente:** Se agregan basándose en el comportamiento de la aplicación y el uso que hacen los usuarios.
 - **Automáticamente:** Se extraen durante el momento de captura (ej. los datos guardados directamente al tomar una fotografía digital).

3. Consulta de Datos Multimèdia Dado que el acceso por contenido es complejo, existen cuatro enfoques para consultar la base de datos:

- **Lingüístico – lingüístico:** Se formulan consultas usando un lenguaje tradicional (como SQL) para buscar coincidencias dentro de los metadatos.
- **Visual – visual:** Acceso directo por contenido, donde se usa un elemento visual (como el esbozo de una imagen) para encontrar imágenes similares en la base.
- **Visual – lingüístico:** Usar una entrada visual (esbozo, colores, píxeles) que el sistema traduce para realizar una búsqueda sobre los metadatos.
- **Lingüístico – visual:** Se expresa la consulta de forma textual y el sistema busca coincidencias en los atributos puramente visuales de las imágenes.

4. Almacenamiento y Tipos LOB (Large Objects) Los datos multimedia se pueden guardar **externamente** a la base de datos o **internamente**. Para el almacenamiento interno, las bases de datos Objeto-Relacionales incorporaron (desde SQL:99) el tipo de dato **LOB**, que se divide en:

- **BLOB (Binary Large Object):** Guarda datos binarios masivos (hasta 4 GB). Es el tipo indicado para **imágenes, vídeos, audios y texto con formato**.
- **CLOB y NCLOB (Character Large Object):** Almacenan grandes cadenas de caracteres (hasta 4 GB). Se utilizan fundamentalmente para texto largo sin formato.
- **BFILE:** A diferencia de los anteriores, no guarda el contenido dentro de la BD. Solo almacena un **apuntador a un archivo físico** del sistema operativo. Su tamaño máximo depende del SO y permite únicamente operaciones de lectura.

Los objetos internos (BLOB, CLOB) tienen la ventaja de que **soportan transacciones** y permiten acceso aleatorio; funcionan a través de un "localizador" (una referencia a dónde están los datos reales).

5. Manipulación y Programación Multimedia La gestión de estos objetos en el servidor (inserción, recuperación y modificación) se lleva a cabo habitualmente a través de las funciones integradas en el paquete **DBMS_LOB**.

- **Carga desde Ficheros:** Podemos cargar una imagen desde un directorio en disco (u Oracle Cloud) apuntándola inicialmente con BFILENAME y volcando su contenido binario en un campo BLOB mediante el procedimiento DBMS_LOB.LOADFROMFILE.
- **Descarga a Ficheros:** Un campo BLOB guardado en la BD se puede exportar de vuelta a un disco físico combinando las lecturas por fragmentos de DBMS_LOB.SUBSTRING y escribiéndolo con el paquete UTL_FILE.

- **Integración en Java (JDBC):** En las aplicaciones cliente escritas en Java, se puede inyectar un BLOB utilizando `PreparedStatement.setBinaryStream()`, o bien recuperarlo para mostrarlo en una interfaz gráfica (ej. usando un `JLabel`) leyendo los bytes mediante `ResultSet.getBinaryStream()` y procesándolo con `ImageIO.read()`.